

Apply-Nav Project Analysis & Architecture Guide

1. Project Overview

Apply-Nav is a locally run, semi-automated LinkedIn "Easy Apply" job application dashboard. It is designed to act as an advanced assistant that seamlessly integrates LLMs (Google Gemini) with browser automation (Playwright/Patchright) to analyze, score, and apply for jobs based on a user's resume.

The project is specifically built with a **Human-in-the-Loop (HITL)** philosophy. It prioritizes account security and precision over pure speed by pausing for manual review, thereby avoiding algorithmic detection and ensuring application quality.

2. Core Architecture

2.1 Backend Server (`job_applier_dashboard.py`)

Built using **FastAPI** and **Uvicorn**, the backend handles both HTTP requests and real-time WebSocket communication.

- **WebSocket Streaming:** Automation progress, logs, status updates, and interactive prompts (screening questions) are streamed continuously to the UI.
- **Shared Session State:** To prevent LinkedIn from detecting simultaneous logins and locking the account, the system maintains a unified, persistent Chromium session (`~/.linkedin-mcp/profile`). The search operation runs headlessly, and the apply operation runs visibly, but both share the same cookie context.

2.2 Frontend UI (`templates/index.html`)

A single-page application built with premium vanilla HTML, CSS (Glassmorphism, dark mode), and JavaScript.

- Uses dynamic layout transitions to display search results.
- Incorporates a real-time terminal overlay to show the user exactly what the background browser is doing.

2.3 LLM Integration (Gemini)

The system leverages Google's Gemini 2.5 Flash model for high-speed reasoning:

- **Job Scoring:** Compares the extracted plaintext resume (`Mohammed_Saifulhuq_Resume.txt`) against the job description. It calculates a compatibility score (0-100), identifies matched skills, highlights skill gaps, and generates a custom outreach note.
- **Screening Question Resolution:** During the automation flow, if a form field requires a non-standard answer, the system extracts the question, options, and resume, passing them to Gemini to suggest an answer. The UI then prompts the user to approve or modify the AI's suggestion.

3. Automation Flow & Logic

3.1 Search & Extraction Phase

1. Connects to the persistent browser context.
2. Uses a custom `LinkedInExtractor` to perform a search query (e.g., keywords, location) specifically filtering for "Easy Apply" jobs.
3. Iterates over the job IDs, scraping job details synchronously to avoid race conditions.
4. Analyzes the job inline using Gemini and broadcasts the results to the UI.

3.2 Easy Apply Phase

1. Validates the session cookies. If redirected to an auth wall, it prompts the user to re-login.
2. Navigates to the specific job page.
3. Detects the correct application flow:
 - Scans for actual "Easy Apply" buttons (`<button>`) or modern SDUI "Easy Apply" anchor tags (`<a>` with `openSDUIApplyFlow=true`).
 - Rejects external apply links.
4. Executes a 15-step multi-page form filling loop:
 - Evaluates standard DOM fields (`input`, `textarea`, `select`, `checkbox`, `radio`).
 - Automatically fills standard PII (Name, Email, Phone, City).
 - Automatically uploads the PDF resume (`Mohammed_Saifulhuq_Resume.pdf`).
 - Resolves custom screening questions using the AI/HITL flow.
5. Pauses execution on the final "Review" page to allow the user to manually click "Submit" (preventing accidental poor submissions).

4. Key Design Decisions for "Better Versions"

Future AI iterations analyzing this codebase should note:

- **Cookie Bridging vs Persistent Context:** The system uses persistent context locking. If deploying to cloud/Docker, cookie bridging (exporting/importing `cookies.json`) is required because cloud instances cannot run stateful profile directories cleanly.
- **DOM Stability:** LinkedIn frequently changes its DOM structure (e.g., changing Easy Apply from a `button` to an `a` tag). Analyzers should prioritize building robust selector heuristics (using ARIA roles or visual locators) rather than hardcoded CSS classes.
- **Rate Limiting Resilience:** Operations intentionally use randomized `asyncio.sleep(random.uniform(x, y))` to simulate human interaction. Future versions should dynamically scale these delays based on HTTP 429 responses.